SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

GLVMS – Guest Lecture & Visitor Management System

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for the Guest Lecture & Visitor Management System (GLVMS). The system digitizes manual visitor entry registers and automates guest lecture scheduling and security workflows

1.2 Scope

The system provides:

- Visitor Registration
- Admin Approval Workflow
- Security Gate Check-in/Out
- Guest Lecture Management
- Equipment Checklist Management
- Reporting & Duration Tracking

The current implementation uses React.js and localStorage as a mock database

This project glvms

2. Overall Description

2.1 User Classes

1. **Visitor**

- Can register visit request
- Can view upcoming lectures

2. **Admin**

- Approves/Rejects visitor requests
- Manages lectures
- Views reports
- Manages checklist

3. **Security**

- Searches approved visitors
- Records Check-in time
- Records Check-out time

2.2 Operating Environment

- Web-based SPA
- Responsive (Desktop & Tablet)
- Built with:
    - React.js
    - Tailwind CSS
    - react-router-dom
    - localStorage

3. Functional Requirements

FR1 – Visitor Registration

- Visitor must provide:
    - Name
    - Phone
    - Department
    - Purpose
- Default status: “Pending”
- Data stored in localStorage

FR2 – Admin Approval

- Admin views Pending visitors
- Admin toggles status:
    - Approved
    - Rejected

FR3 – Gate Management

- Security searches by phone number
- Security records:
    - checkInTime
    - checkOutTime
- System stores timestamp

FR4 – Lecture Management

- Admin can:
    - Add Lecture
    - Edit Lecture
    - Delete Lecture
- Lecture includes:
    - Topic
    - Speaker
    - Date
    - Equipment checklist

FR5 – Reporting

- Admin can see:
    - Visitor duration
    - Total visitor count
    - Entry/Exit logs

4. Non-Functional Requirements

Security

- Role-Based Access Control (RBAC)

This project glvms

- Admin and Security dashboards protected

Performance

- Search and filtering < 200ms
- No page reload (SPA architecture)

Usability

- One-click actions
- Minimalist UI
- Mobile responsive

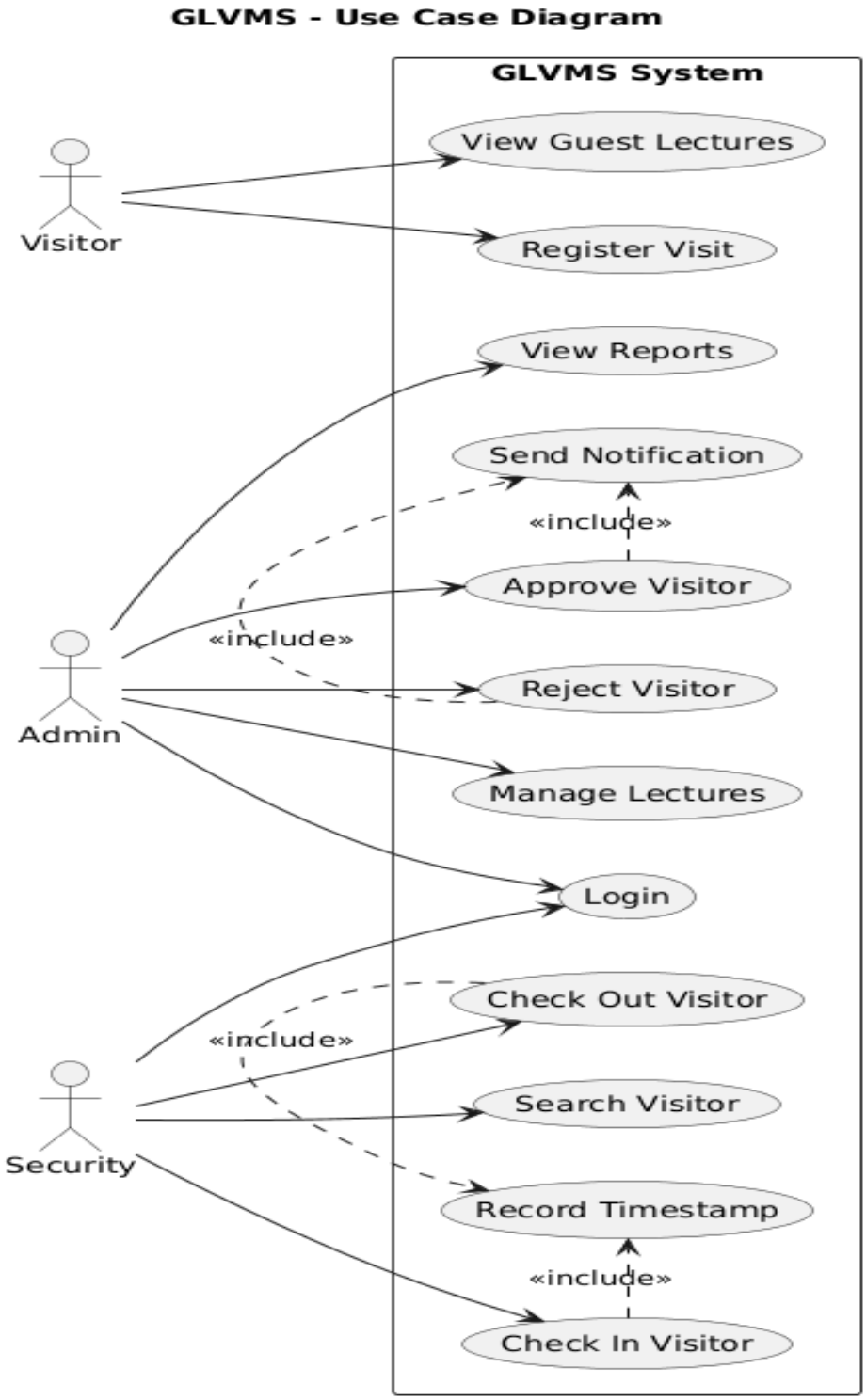
Reliability

- Data persists after browser refresh
- No data loss unless localStorage cleared

5. Use Case Diagram (For SRS)

Actors:

- Visitor
- Admin
- Security



6. Database Schema Design (Future Backend Version)

Currently localStorage is used

This project glvms

But if converted to real database, these tables will be created:

Tables Overview

Total Tables: **5**

1. Users
2. Visitors
3. Lectures
4. LectureItems
5. EntryLogs

1. Users Table

Field	**Type**	**Description**
id	INT (PK)	Unique ID
username	VARCHAR	Login username
password	VARCHAR	Hashed password
role	ENUM	admin / security

2. Visitors Table

3. EntryLogs Table

Field	Type
id	INT (PK)
visitor_id	INT (FK)
check_in	DATETIME
check_out	DATETIME

Relationship:
Visitor (1) —— (Many) EntryLogs

4. Lectures Table

Field	Type
id	INT (PK)
topic	VARCHAR
speaker	VARCHAR
date	DATE
description	TEXT

5. LectureItems Table

Field	Type
id	INT (PK)
lecture_id	INT (FK)
item_name	VARCHAR
ready	BOOLEAN

Relationship:
Lecture (1) —— (Many) LectureItems

- **Database Relationships Summary**

Users (1 role-based login)
Visitors (1) —— (Many) EntryLogs
Lectures (1) —— (Many) LectureItems

7. Page-wise Documentation

1. Home Page

- Navigation bar
- CTA buttons
- Overview of system

2. Visitor Registration Page

- Form submission
- Data saved to localStorage
- Status: Pending

3. Login Page

- Role-based login
- Redirects to:
    - Admin Dashboard
    - Security Dashboard

4. Admin Dashboard

Tabs:

- Visitors
- Lectures
- Reports

Functions:

- Approve/Reject
- Add/Edit/Delete Lecture
- View Duration Reports

5. Security Dashboard

Features:

- Search visitor by phone
- Check In
- Check Out
- Real-time status update

6. Guest Lectures Page

- Public lecture display
- Sorted by date
- Card-based UI

8. Future Enhancements

- Backend (Node.js + MongoDB)
- SMS integration
- QR Code entry system
- Cloud deployment
- Analytics Dashboard